Лабораторная работа №3. Backend на Java с авторизацией

Цель

В рамках лабораторной работы необходимо перенести backend системы управления студентами из лабораторной работы №2 на Java и развить его до полноценного серверного приложения с хранением данных, пользователями и разграничением доступа.

Основная бизнес-логика предыдущей лабораторной работы должна быть сохранена. Приложение по-прежнему должно предоставлять REST API для работы со студентами, однако теперь данные должны храниться в реляционной базе данных, а доступ к операциям должен зависеть от авторизации и роли пользователя.

Варианты реализации

В лабораторной работе предусмотрено два основных варианта реализации:

Вариант 1 — Spring Boot

Backend реализуется на базе экосистемы Spring Boot.

Вариант 2 — Java EE / Jakarta EE

Backend реализуется средствами Java EE / Jakarta EE.

Дополнительно преподаватель указывает технологию взаимодействия с базой данных для конкретного варианта задания.

В качестве технологии доступа к данным могут использоваться, например:

- Hibernate / JPA;
- EclipseLink / JPA;
- jOOQ.

Таким образом, конкретный вариант лабораторной работы определяется сочетанием серверной платформы и технологии работы с базой данных.

Работа со студентами

Необходимо сохранить REST API, разработанный в предыдущей лабораторной работе. Приложение должно поддерживать получение списка студентов, получение отдельного студента, создание, полное и частичное изменение, а также удаление.
При работе со списком студентов необходимо сохранить возможность поиска, фильтрации, сортировки и пагинации.
Параметры запроса и формат взаимодействия должны быть организованы таким образом, чтобы API оставался удобным для использования клиентским приложением.

Хранение данных

В отличие от предыдущей лабораторной работы данные должны храниться в реляционной базе данных.

Для взаимодействия с базой данных необходимо использовать технологию доступа к данным, указанную преподавателем в варианте задания. В качестве такой технологии могут использоваться Hibernate / JPA, EclipseLink / JPA, jOOQ или другой согласованный инструмент.

Пользователи и авторизация

В системе необходимо добавить работу с пользователями.

Пользователь должен иметь как минимум уникальное имя, электронную почту, пароль и роль.

Регистрация должна создавать нового пользователя только после проверки корректности введенных данных.

Пароли не должны храниться в открытом виде.

После успешной авторизации пользователь должен получать данные, необходимые для дальнейшего обращения к защищенным ресурсам.

JWT

Для авторизации необходимо использовать JWT.

Механизм авторизации должен поддерживать access- и refresh-токены и позволять пользователю продолжать работу без повторного ввода логина и пароля при истечении access-токена.

Роли и разграничение доступа

В системе необходимо предусмотреть как минимум две роли:

- ADMIN
- USER

Пользователь с ролью `ADMIN` должен иметь возможность выполнять все операции со студентами.

Пользователь с ролью `USER` должен иметь возможность просматривать список студентов и получать информацию об отдельном студенте, но не изменять данные.

Разграничение прав должно выполняться непосредственно на backend.

Архитектура приложения

Приложение должно иметь понятное разделение ответственности между его частями.

Работа с HTTP-запросами, бизнес-логика, доступ к данным и представление данных не должны быть смешаны в одном компоненте.

Для передачи данных через REST API необходимо использовать DTO либо аналогичный подход, отделяющий внешний контракт API от внутренней модели приложения.

Конкретная структура пакетов и классов определяется разработчиком.

Валидация и обработка ошибок

Необходимо сохранить серверную валидацию, реализованную в предыдущей лабораторной работе, и расширить её с учётом появления пользователей.

Backend должен проверять корректность поступающих данных, ограничения модели и уникальность тех полей, для которых она необходима.

Ошибки должны обрабатываться централизованно и возвращаться клиенту в едином формате.

API должен использовать HTTP-статусы в соответствии с результатом выполнения запроса.

Документирование и проверка API

REST API необходимо документировать с использованием Swagger / OpenAPI либо совместимого инструмента.

Также необходимо подготовить коллекцию запросов для Postman или Bruno, позволяющую продемонстрировать основные сценарии работы приложения:

- работу со студентами;
- регистрацию;
- авторизацию;
- обновление токена;
- работу пользователей с различными ролями.

Конфигурация и логирование

Приложение должно содержать серверное логирование, достаточное для анализа его работы и возникающих ошибок.

Конфигурация, зависящая от окружения, должна быть вынесена за пределы исходного кода.

В проекте необходимо предоставить `.env.example` либо аналогичный пример конфигурации, позволяющий понять, какие параметры требуются для запуска приложения.

Вопросы к защите

1. Java в серверной веб-разработке. Особенности языка и платформы JVM.
2. Spring Boot и Java EE / Jakarta EE: основные отличия и подходы к разработке.
3. ООП в Java: классы, интерфейсы, наследование и полиморфизм.
4. Аннотации в Java и их использование в серверных приложениях.
5. Dependency Injection и Inversion of Control.
6. Архитектура backend-приложения. Разделение ответственности между слоями.
7. Исключения в Java и централизованная обработка ошибок.
8. Реляционные базы данных. Таблицы, строки, столбцы, первичные и внешние ключи.
9. Связи между сущностями в реляционной базе данных: один-к-одному, один-ко-многим, многие-ко-многим.
10. ORM: назначение, преимущества и особенности отображения объектов на реляционную модель.
11. Транзакции в базе данных. Основные свойства транзакций и их применение в backend-приложениях.
12. Аутентификация и авторизация. Различия между ними.
13. JWT: устройство токена, подпись, access- и refresh-токены.
14. Ролевая модель доступа и защита серверных ресурсов.
15. Хеширование паролей. Назначение соли и применение алгоритмов типа BCrypt.
16. Хранение конфигурации, секретов и параметров окружения в backend-приложении.